Fervo: building blocks for an agent-native code platform

Problem and vision

Coding agents now write much of the world's new code, yet they still ship it through GitHub, a platform built for humans that i) have all the necessary context to prioritize pull requests and know who own what and ii) review one pull request at a time. For modern multi-agent coding workflows, these two points are major bottlenecks for the following reasons:

1. The agents are blind until a feature is committed to the repo: none of them sees what the others are trying or what already failed, so they repeat directions and drift apart. Moreover, the same context needs to be rediscovered over and over wasting tokens. In one Anthropic experiment, 18 of 30 agents created a git branch with the exact same name.
2. PRs are bigger and a lot more frequent, but today most enterprises and small teams still have a human reviewing the diffs or an AI generated summary of the PR before merging them. This is due to two main factors:
    - agents are bad at supervision and prioritization since they are often not aware of ongoing uncommitted work and long term goals. In particular, changes to the original architecture are very frequent and grow with the number of PRs, and engineering plans and PRDs in .md file quickly go stale.
    - agents grade their own work when writing a PR description, while the only independent check and supervision left is either from a person reading every diff or another fresh agent that does not have all the necessary context about everything going on in the repo to properly judge the contribution.

    Anthropic, whose merged changes per day grew from about 2,500 to over 30,000 in four months, calls developer infrastructure "built for human scale" the new bottleneck.

Fervo is a platform made for coding agents (Claude Code, Codex, pi, OpenClaw) sharing a repository. Every feature is optimized for agentic use and reduced latency, by providing all the necessary context for agents to scope their work and make their changes merge quickly.

The audience are solo developers and researchers running 2 to 10 agents, and small teams that mix few human pull requests with heavy use of agent swarms.

Product and agent interaction

Fervo takes the same minimal philosophy as pi, the coding agent that ships with four tools and a short prompt and leaves the rest to extensions, and does the same for repo management. It is a CLI and a Python SDK that sits on top of git, not a hosting service. `fervo serve` turns a machine the team controls into a git remote: a bare repository plus one SQLite file.

Then every team assembles its own platform from the same core blocks:

- A shared frontier. One map of every direction tried by other agents, how it scored and what is still open. A change moves the frontier only when its gain beats measurement noise.
- Declarations. Each agent states what it will do before it starts, so others steer around it.
- Independent judgment. No agent scores its own work. Predefined tests, a judge from a different provider and agents from other providers do.
- A human override. An admin can merge, veto or revert anything leaving a trace.

These primitives are handled by this provided machine: it computes context, records declarations, keeps a scored frontier of work in progress, evaluates every change independently, and merges by policy. Everything else, from continuous integration to review rules and the interface, is an optional extension left to the developers to decide.

Any agent that works on the repo (ClaudeCode, pi, OpenClaw, Codex, Hermes) calls the CLI, which returns JSON, and learn it from a SKILL.md file that their harness loads. Every LLM call runs in `fervo runner`, a process the operator runs with their own API keys.

To ensure backward compatibility, `fervo import` moves a GitHub repository, with its history and open pull requests, onto the team's remote. We reuse git, the harnesses and the operator's model providers.

Example: four agents improve a Python parsing library. The goal is "a streaming parser API", the metric is benchmark throughput, and the tests must pass. An agent runs `fervo context --json` and gets the goal, the best current nodes with their scores, the open claims and the directions already tried, all from database queries. It declares `fervo claim --parent n_42 --scope src/lexer/ --intent "Building an incremental tokenizer, keeping public API"` and gets a list of blocked files plus a soft warning that codex-1 holds an overlapping claim, so it can narrow its scope or abandon the task. When its commit is ready, `fervo submit --lease l_9 --key k1` returns a job. The runner, not the author, runs the tests and three paired benchmark runs of the node and its parent on held-out inputs. It then asks a judge from another provider to compare the node with its parent three times in each order. `fervo status j_31` then returns `{"status":"verified","node":"n_57","outcome":"improved","delta":0.09,"interval":[0.04,0.14],"rank":1}`. On `within_noise` the agent tries something else, and the node stays visible for others to build on. On `rejected` it reads the reason, such as `touched pinned path bench/`. If the policy is met (tests pass, the score improves, one passing review comes from each model provider in use), Fervo merges the node into `main`.

On top of this, other useful agent-first features could be:

- Auto issue creation and resolution loops
- Auto-compact for repo memory and detailed history creation
- Auto context-aware bug scan
- Auto documentation updater

Technical design

Five kinds of object live in SQLite: actors (a token with a kind, provider and model), claims, nodes (submitted commits with their scores), verdicts (who checked a node, the result, the evidence and the hash of the prompt or command used), and a job queue for the runner. The policy is one `fervo.toml` file in the repo. The frontier, overlaps and tried directions are queries recomputed on every write, so the server never calls a model. A node's gain over its parent combines two parts:

- the paired difference in each metric, measured on the same runs and normalized against a baseline measured several times at setup;
- a pairwise judge preference. The judge always comes from a provider other than the author's, runs in both orders to cancel position bias, and sees the goal, the declaration, the diff and the measured results. It treats code as data.

A node joins the frontier only if the interval on that paired gain lies above zero. Nodes within the noise stay visible as stepping stones, and "tried" labels expire, so the search does not collapse into hill climbing. Agents push only to their own branches, and only the policy engine or a human moves `main`. Verdicts come only from the runner or a human, never from the author, and reviews count once per provider. That is a mitigation rather than a guarantee, since models from different providers still share errors, so measured metrics always keep a weight. Agents see what is scored but not how: the judge prompt and the held-out inputs live with the runner, and edits to the benchmark or `fervo.toml` are rejected. Every write carries an idempotency key, a crashed evaluation becomes a `failed` node that keeps its log, and jobs wait while the runner is offline.

Prototype and checks

The smallest demo imports a small open-source Python library that has tests and a CPU benchmark, and runs four agents (two Claude Code, Codex, pi) on one goal, with the server and runner on a laptop. We compare it with two baselines at equal tokens and time:

- the same agents on GitHub (pull requests, AGENTS.md, a shared notes file, human merges);
- the same agents working independently, with the best result picked at the end.

We measure the metric reached, duplicated directions (labelled independently of Fervo), regressions merged into `main` (found by held-out tests the agents never see), human review minutes, and the tokens agents spend getting oriented. Success means at least the same metric, fewer duplicates, and no held-out regressions.

Team members and feasibility

- Filippo Strub: built an ontology-based data platform for agents at Philogen, a biotech firm, and worked as a software engineer for 8090 solutions, a statup building an automated Software Factory.

- Roshan Patel: 4 years as a software engineer at JPMorgan Chase and a background in computer science and data science.

We split the work into the remote side (storage, permissions, policy, runner, scoring) and the agent side (CLI, SKILL.md, import, interface, study). Both of us build, and roles can shift.

We scope the project to be an MVP, with minimal features to be actually useful. We believe that with our prior coding and backend developement experience the project is feasible in 4 weeks.

Feedback

Six AI reviewers covered the first iteration of the plan: an adoption skeptic, a prior-art hunter, an HW5 grader, a Codex agent in a long loop, a researcher running five agents and an open-source maintainer reviewed it in role.

Prior art narrowed our claim to the scored frontier, strict independence and evaluation on the operator's keys. Other main changes were: Github compatibility command, independent judge design, idempotency key for writes, scoping the smallest demo to fit in 4 weeks.

This second iteration was passed trough a literature and landscape reviewe (about 40 papers, 35 products, 39 open-source repos), that led to these changes:

- judge from a provider other than the author's, comparing pairwise
- paired-difference promotion
- held-out scoring inputs
- expiring "tried" labels
- best-of-N baseline

One assumption remains to check: that operators will let policy-approved merges land while reviewing only intent and verdicts. We will ask three people who run agent swarms, and in the demo a human will audit every approved merge.

Project listing

61: Fervo, minimal repo management for agent contributors. Both members are marked "in team" in the Students tab.